3. Beadandó dokumentáció

BHYUHA – Sándor Kolos Barnabás

1.fejezet

Feladat:

Kezdetben egy 2 egység (fej és csörgő) hosszú csörgő kígyóval kell felszednünk a sivatagos játéktéren megjelenő élelmet. A játéktéren egyszerre 1 elemózsia lehet véletlenszerűen elhelyezve olyan mezőn, melyen nem a kígyó található. A kígyó a játéktér közepéről egy véletlenszerűen választott irányba indul. A továbbiakban a felhasználó a billentyűzet segítségével válthat majd irányt. Élelemhez érve, a kígyó mérete egy egységgel nő. A játékot nehezítse, hogy a sivatagban kövek is találhatók melyeknek, ha nekimegy a kígyó, akkor véget ér a játék. Abban az esetben is elveszítjük a játékot, ha a kígyó saját magának megy neki, vagy a pálya szélének. Ezekben az esetekben jelenjen meg egy felugró ablak, melyben a játékos a nevét megadva el tudja menteni az adatbázisba az eredményét, mely a játék során a kígyó által elfogyasztott élelem összege. Egy menüpontban legyen lehetőségünk a 10 legjobb eredménnyel rendelkező játékost megtekinteni, az elért pontszámukkal, továbbá lehessen bármikor új játékot indítani egy másik menüből.

2.fejezet

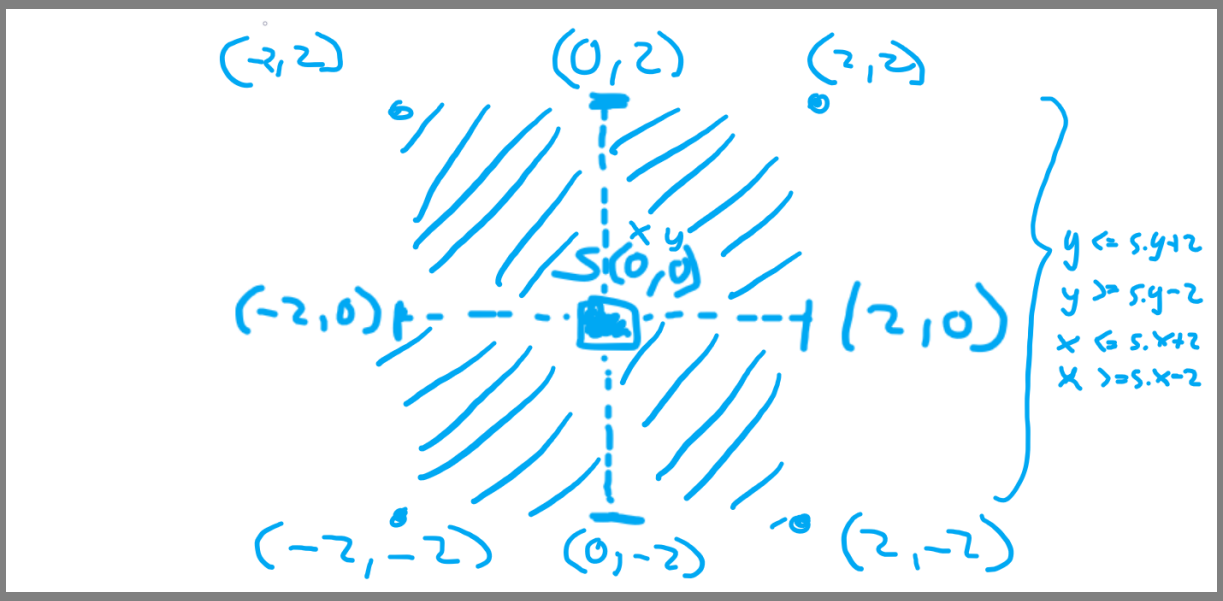
Rövid elemzés

Egyre mélyebb zugait fedezzük fel a Javanak, most már egy highscore rendszerrel felszerelt, billentyűzettel irányítható játékot elkészítve. Habár sqlt biztos kell majd használni és ezt az oldalát a dolognak még mindig nem értem hogyan működik az órai fájlok biztos jól jönnek majd. Itt sokkal több részre kell bontanom majd a feladatot, ha az objektum orientáltságát meg akarom tartani, mint az előző beadandóban.

Megoldás terve

Közelebbről megnézve a feladatot fogalmam sincs hogyan kéne a pontrendszert működtetnem, úgyhogy azt kell felhasználnom, amit órán mutatott a gyakvez (ön). Ehhez sajnos az alapvető ötleteim egy részét el kell vetnem és olyan struktúrával megalkotnom a programot, mint ami a példában szerepelt, nehogy valami ne működjön, viszont nem fogom teljesen kihagyni belőle a saját megoldásaim, mégiscsak az a legjobb része.

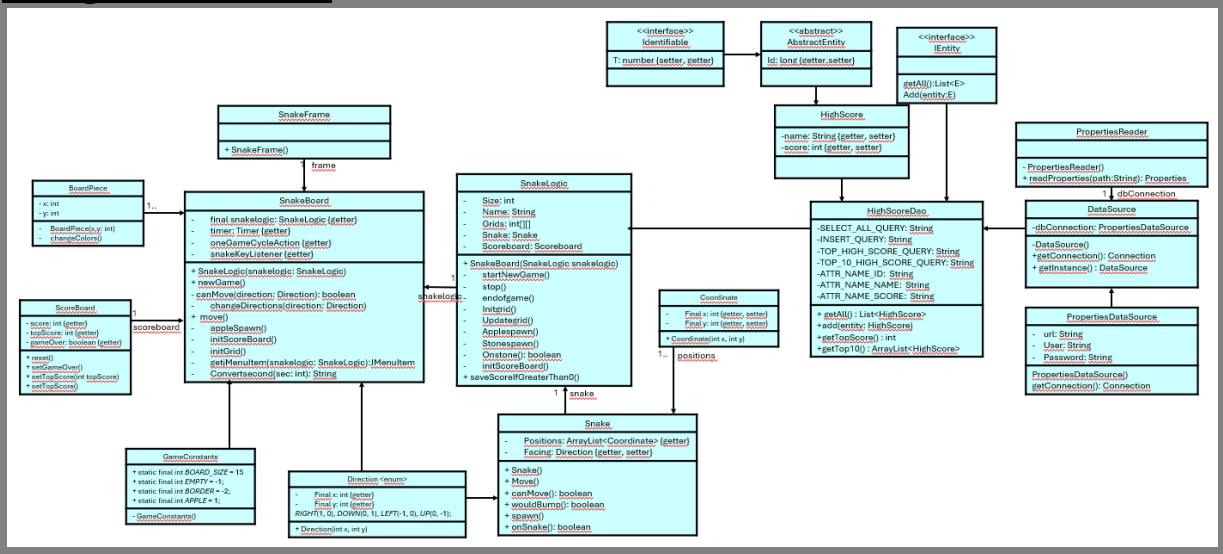
A packageek felépítése tehát lényegiben ugyanaz lesz, de a bennük működő független függvények kitalálása már rám vár.
(végül sokkal jobban megértettem a databázis működését, úgyhogy happy end)



spawnStones függvény megtervezése (az ábra mutatja, hogy a megszületett kő körül hol nem születhet le egy újabb kő)

3.fejezet

Megvalósítás:



Ahogy az ábra mutatja, az osztályok melyeknek a működésével tisztában voltam teljesen kevesen voltak, de ezekben is van elég mélység, hogy lehessen róluk beszélni. Kedvenceim:

- Move(): a játékmenet fő elemei ebben a függvényben jelennek meg, itt dől el, hogy vesztesz, vagy növeled-e a pontszámod, e körül forog igazán az egész program.
- SnakeBoard()/OneGameCycleAction: kicsit csalás konstruktort megnevezni, mivel olyen ez, mintha az egész osztályt nevezném kedvenc függvényemnek, viszont ebben a konstruktorban inicializáljuk a timert, ami lefuttatja a függvényt amire valójában a fókuszt helyezném, a oneGameCycleAction actionPerformed függvényét, ami szintén óriási szerepet foglal el a játék felépítésében.

Enélkül akkor történne bármilyen változás a játéktéren amikor a játékos önszántából mozogna. Így szorítja az idő, a játékba hozva azt a szükséges nehézségi elemet.

4.fejezet

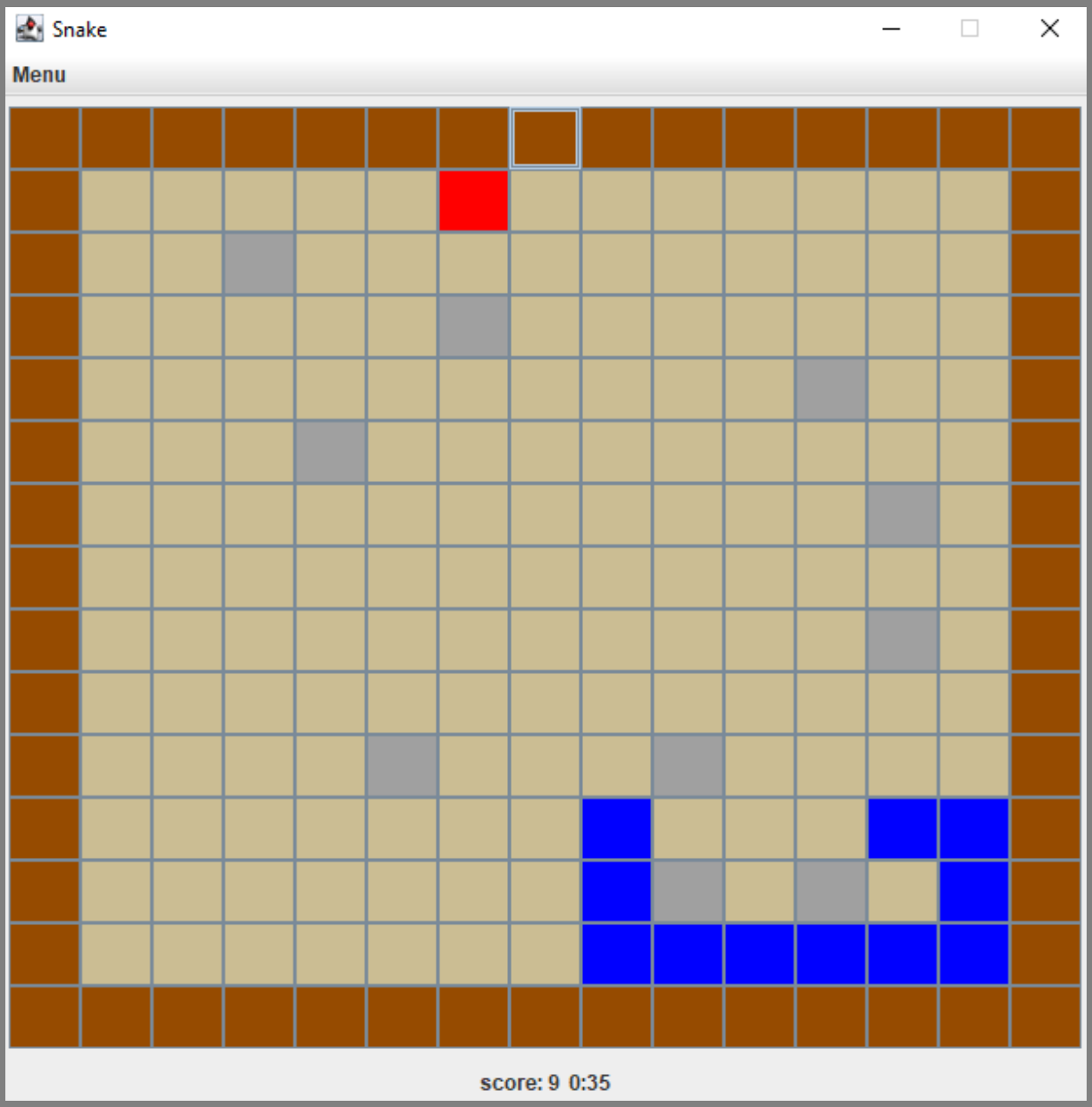
As a	tryhard játékos	
I want to	megnyerni a játékot	
1.	GIVEN	elindítjuk a játékot
	WHEN	eljut 10 pontig
	THEN	meghal a falban
2.	GIVEN	elindítjuk a játékot
	WHEN	eljut 10 pontig
	THEN	meghal a farkában
3.	GIVEN	elindítjuk a játékot
	WHEN	0 pontot ér el
	THEN	elbénázta, a pontjai fel se kerülnek a táblára
4.	GIVEN	elindítjuk a játékot
	WHEN	előző menet miatt dühösen játszik
	THEN	megint meghal, mivel a heves érzelmek miatt nem figyelt eléggé
5.	GIVEN	elindítjuk a játékot
	WHEN	ragequitel mielőtt megadná a nevét
	THEN	bezáródik az ablak
As a	chill játékos	
I want to	jól szórakozni	
6.	GIVEN	elindítjuk a játékot
	WHEN	eljut 5 pontig
	THEN	meghal a falban, de nem zavartatja magát, mert egy chill guy
7.	GIVEN	elindítjuk a játékot
	WHEN	eljut 6 pontig
	THEN	meghal a farkában, de nem zavartatja magát, mert egy chill guy
	GIVEN	elindítjuk a játékot

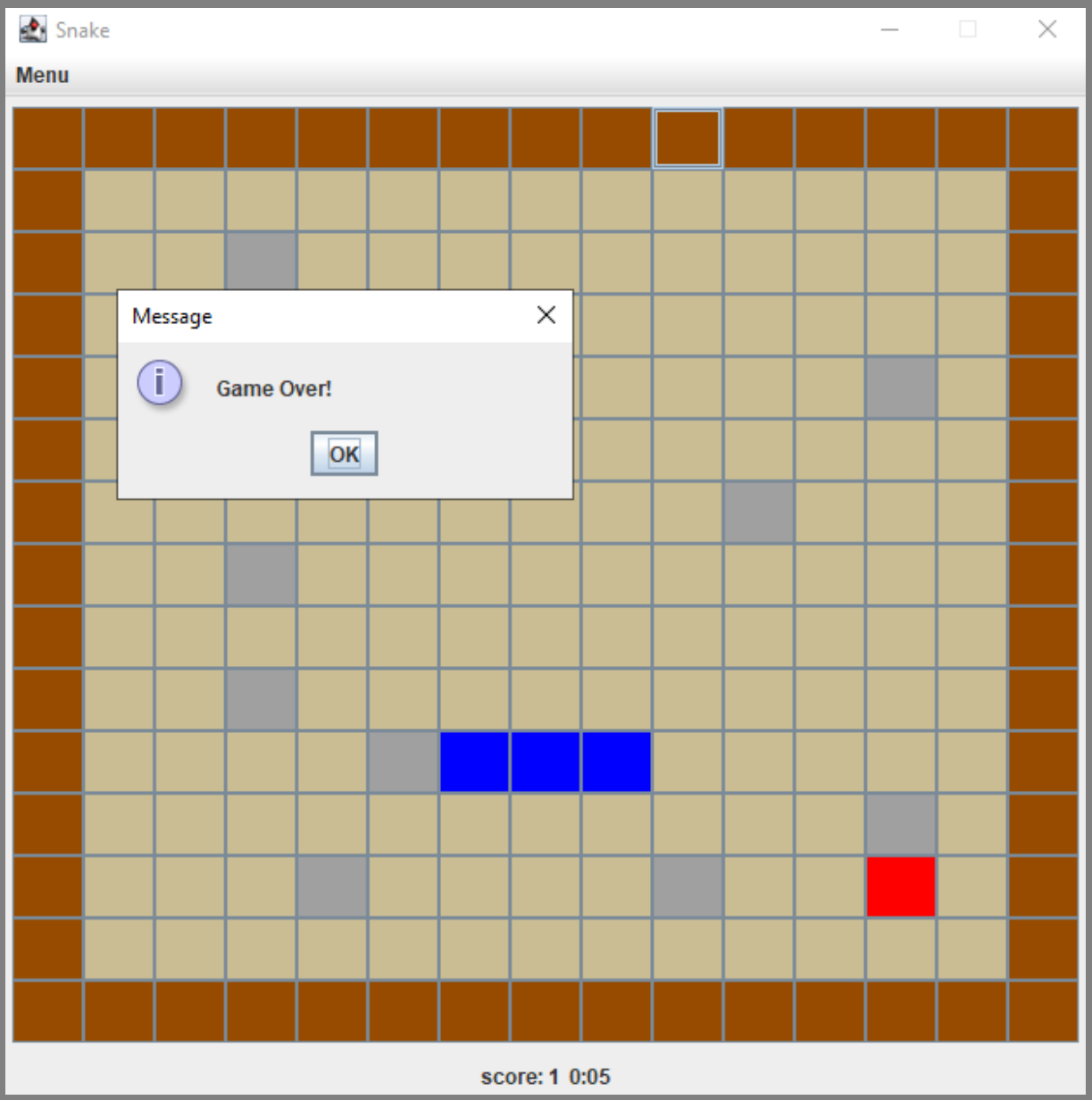
8.	WHEN	rosszul adja meg a nevét
	THEN	tovább játszik, nem zavartatja magát, mert egy chill guy
9.	GIVEN	elindítjuk a játékot
	WHEN	nem tudja felvenni az almát, mivel nehéz helyen van, inkább indít egy új játékot
	THEN	elindul a játék egy újabb kiosztással, nem érdekli, hogy egy pontot sem ért el, mivel ő egy chill guy

10.	GIVEN	elindítjuk a játékot
	WHEN	anyja hívja ebédelni
	THEN	kilép mielőtt megadhatná a nevét, mivel ő egy chill guy és anya fősztje fontosabb, mint a játék
As a	programozó	
I want to	a készítő agyára menni	
11.	GIVEN	elindítjuk a játékot
	WHEN	nehéz helyen van az alma, megpróbál egy algoritmust írni az almák megszerzésére
	THEN	sikeresen elrontja magának a játékélményt
12.	GIVEN	elindítjuk a játékot
	WHEN	az algoritmusa fut, de rosszul írta meg!!
	THEN	kicrashel a játék (O o O)
13.	GIVEN	elindítjuk a játékot
	WHEN	megpróbálja fixálni az elgoritmus siker nélkül, most már a nevét se tudja megadni
	THEN	az algoritmus azonnal bezárja a játékot
14.	GIVEN	elindítjuk a játékot
	WHEN	megpróbálja magától megoldani, nehezen, de megpróbálja élvezni (allergiás a brute forcera)
	THEN	rájön, hogy ez egy szuper játék, és legközelebb nem kéne a készítőket stresszelni a butaságaival
	GIVEN	el sem indítja a játékot

15.	WHEN	megkíméli a készítőt több user story írásától
	THEN	a játék nem indul el (Happy End)

képernyőképek futás közben:




Snake
Menu
Message
Game Over!
OK
score: 1 0:05

Események:

- Játék kezdése: SnakeBoard: startnewgame() -
  új táblát rajzol, elindítja a játékot a snakelogicon keresztül és elindítja a timereket amik mérik az időt és a pontszámot
- WASD nyomása: SnakeBoard: snakeKeyListener: az alapján, hogy melyik gombot nyomtuk le a snakelogicon keresztül csekkolja, hogy az irány amerre menni akarunk érvényes-e, majd ha igen akkor arra fordítja a kígyót
- játékciklus: SnakeBoard: oneGameCycleAction – ezalatt megint a snakelogicon keresztül mozog a kígyó, itt nézzük meg még mozgás előtt, hogy mi fog történni, majd a táblán megjelenítjük az eredményt
  egy pár lehetséges esemény:
  - ütközés: nekimentünk egy kőnek vagy kerítésnek, ilyenkor vége a játéknak
  - almaevés: felvettünk egy almát, kapunk egy pontot és nő a kígyónk
  - üres mező: lépünk egyet, ilyenkor semmi speciális esemény nem történik
  (mindezek az események a snakelogic által vannak lekezelve)
- menügombok: SnakeBoard: menuBar -
  különböző menügombok állnak segítségünkre játék közben:
  - new game: elindítja a startnewgame() függvényt miután törölte az aktuális táblát
  - top 10: kilistázza a 10 legnagyobb pontszámot elért játékost és pontszámuk (databázison keresztül)
  - exit: bezárja a programot
- Játék vége: Kiírja egy kis üzenetben, hogy game over, de nem zárja be az ablakot, engedi, hogy új játékot indíts a menüből (SnakeBoard kezelésével)